Föreläsning 8

Mängd, Avbildning, Hashtabell

Föreläsning 8

- Mängd (Set)
- Avbildning (Map)
- Hashtabeller
- Hashkoder
- Öppen adressering
- Länkning
- Implementering

Mängd (Set)

En mängd är en samling som inte innehåller några kopior

lägger man till A till {A,B,C} får man {A,B,C}

Operationer:

- test för medlemskap
- lägga till element
- ta bort element
- union
- snitt
- differens
- delmängd

Inom datalogin pratar vi också om ordnade mängder

Mängd i Java API (Set)

Set är ett interface

Implementeras av tex TreeSet (också ordnad mängd) och HashSet.

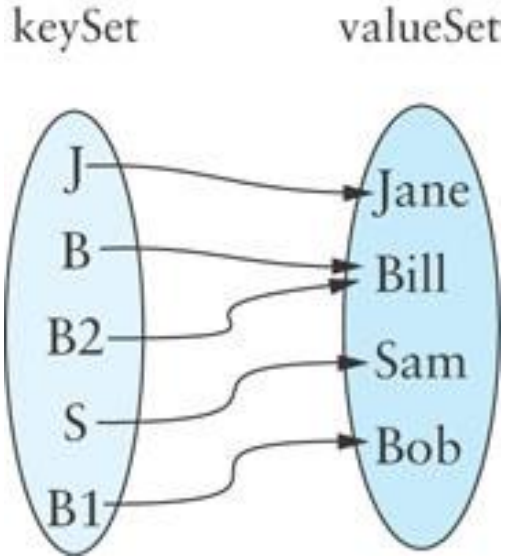
Det finns också ett BitSet där man mycket effektivt kan lagra vilka heltal som ingår i en mängd av heltal. En bit per heltal.

Avbildning (Map)

En avbildning är en mängd av ordnade par (nyckel, värde)

Nycklarna är unika men värdena behöver inte vara det

Har metoder såsom get(key) och put(key,value)



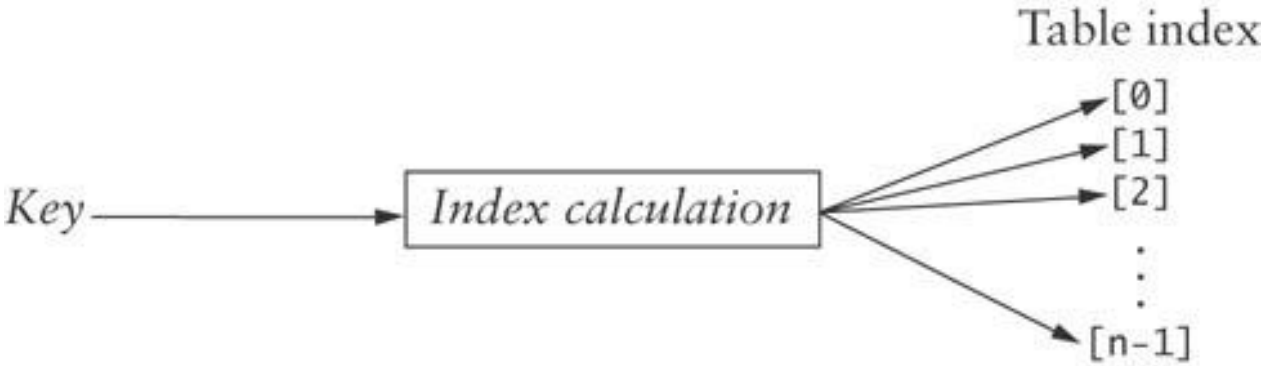

```
{(J, Jane), (B, Bill),
 (S, Sam), (B1, Bob),
 (B2,  Bill)}
```

Hashtabeller

Kan användas för att implementera en mängd eller avbildning

Erbjuder access via nyckel (ej position) i genomsnitt med effektiviteten: O(1)

Principen: Nyckeln transformeras till ett heltal (hashkoden) som transformeras till ett tabellindex:



Nyckeln direkt skulle normalt ge för stor tabell men är effektivt.

Nyckel%Tabellstorlek ger normalt ej jämn distribution så vi får onödigt många kollisioner (olika nyckel samma index)

Generera hashkoder

Nyckeln består normalt av en textsträng eller ett antal siffror (personnummer, emailaddress, osv)

Antalet möjliga nycklar är mycket större än tabellstorleken

Typiskt experimenterar man för att få fram bra hashkoder

Målet är en jämt fördelad slumpmässigt distribution över tabellen

Enkla algoritmer genererar onödigt ofta kollisioner

Algoritmen måste vara effektiv. Målet är ju att undvika O(n) sökning.

Java string-hashkoder

public int hashCode()
Returns a hash code for this string. The hash code for a String object is computed as
s[0]*31^(n-1) + s[1]*31^(n-2) + ... + s[n-1]

s[i] är då heltalsvärdet för i:te tecknet

“Cat” blir då:

$$‘C’ \times 31^2 + ‘a’ \times 31 + ‘t’ = 67{,}510$$

31 är valt för att det är ett lagom stort primtal för att generera få kollisioner

För att beräkna s index I en tabell: `s.hashCode() % table.length`

Metoden lyckas generera koder ganska jämnt distribuerade över intervallet så sannolikheten för kollisioner blir proportionell mot hur full tabellen är.

Hashkoder i egna klasser

Alla klasser ärver

```
public int hashCode()
```

från object.

Den genererar samma hashkod om två objekt är lika enligt equals och använder sig av den interna adressen på heapen. Normalt vill man precis som med equals implementera en egen (override). I princip ska man alltid då man skriver en egen equals också implementera hashCode så att två objekt som är lika också får samma hashkod.

Enklaste sättet att skapa en hashkod i egna objekt är att använda:

```
public static int hash(Object... values)
```

i java.util.Objects.

Där man skickar in alla medlemmar som man jämför i den equals man implementerar.

Hantering av krockar i hashtabeller

- Öppen adressering
- Länkning (Chaining)

Öppen adressering

Vid insättning:
Om index beräknat med ett elements nyckel ger null har vi hittat positionen
Om index ger element med annan nyckel så ökar vi index med 1
Fortsätt öka index med ett tills vi får null (förutsätter att tabellen ej är full)

Vid sökning:
Om index beräknat med ett elements nyckel ger element med nyckeln har vi hittat elementet
Om index ger element med annan nyckel så ökar vi index med 1
Fortsätt öka index med ett tills vi hittar elementet med rätt nyckel eller null då elementet saknas

(kommer vi till slutet börjar vi från början %tabellängd)

Vid kollision påbörjas en linjär sökning. Den ska helst inte bli för lång för då får vi O(n). Viktigt att ha väl tilltagen tabellstorlek.

Sökning

Hur vet vi när vi ska sluta leta om tabellen är full:

- sluta när ökningen av index är tillbaka vid startvärdet
- tillåt aldrig en tabell bli full utan öka storleken när en viss del är full

Obs att det är en mycket dålig ide att tillåta full/nästan full array då vi kommer att få $O(n)$ när vi söker efter något som inte finns i tabellen.

Ta bort element

Vi kan inte bara sätta indexplatsen till null för andra element som kolliderat kommer då inte att hittas

Istället lagrar vi en dummy

Dock går den lediga platsen inte att fylla utan att kontrollera att elementet inte finns längre ned pga en tidigare kollision

Minska antalet kollisioner genom att

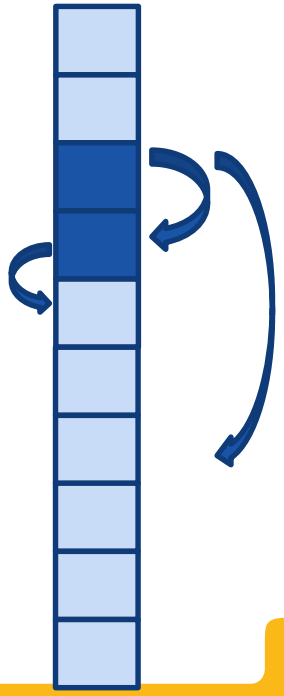
Använd ett primtal för tabellens storlek

-Om en hashkodalgoritm ger att data med tex samma första bokstäver ger samma slutsiffror i hashkoden kommer modulo 1000 ge en krock. Använder vi ett primtal minskar risken.

Öka storleken när den är för full. OBS alla element behöver sättas in igen (rehash). Sätt inte in dummy's.

Använd kvadratisk sondering vid krock

Kvadratisk sondering vid krock



När vi använder öppen adressering bildas kluster av poster. Om vi får en krock har vi två poster i rad. Sannolikheten att posten efter dessa ska besättas är nu 3 gånger så stor som att en ensam post ska besättas (tre index resulterar i att den besätts).

Vid kvadratisk sondering gör vi istället följande. Vid krock titta på nästa index – om detta är upptaget titta 4 index bort om det också är upptaget titta 9 index bort osv: 1, 2^2, 3^2 ,…

3 och 4 är upptagna: 3 ger 3+4=7, 4 ger 4+1=5 – ingen klustring!

Indexberäkningen kan göras effektiv (k=-1):

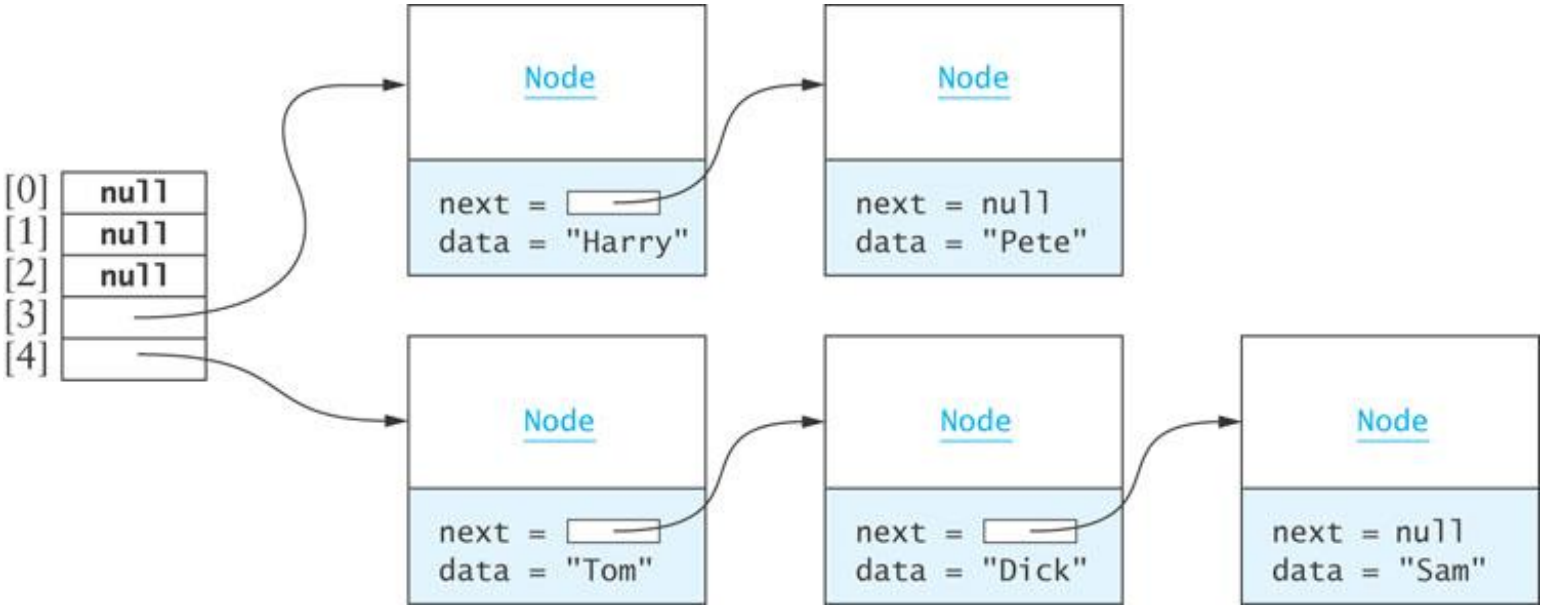
```
k+=2;index=(index+k)%table.length
```

Kan dock missa tomma platser och hamna I oändlig loop. Händer dock aldrig om tabellen är maximalt halvfull och storleken är ett primtal.

Länkning

Varje plats i tabellen refererar till en länkad lista med de element som har hashkod för detta index.

Listan kallas bucket (hink) och metoden bucket hashing



Länkning – öppen adressering

- Endast värden med samma hash->index genomsöks
- Ökar storleken dynamiskt utan rehash. Dock brukar man försöka hålla den mindre än 75% full (*) och om den blir för full expanderar man och gör rehash.
- När du tar bort ett element behövs ingen dummy
- Tar mer plats pga länkarna. Tomma platser tar inte mer plats men varje full plats har en referens extra (förutsätter enkellänkad lista utan size-variabel).

* Jag och java gör rehash vi 75% beläggning, boken vid 300%

Jämförelse

L	Number of Probes with Linear Probing	Number of Probes with Chaining
0.0	1.00	1.00
0.25	1.17	1.13
0.5	1.50	1.25
0.75	2.50	1.38
0.85	3.83	1.43
0.9	5.50	1.45
0.95	10.50	1.48

L - Load factor är antalet fyllda platser / tabellstorlek

Hashtabell – sorterad arraylist- binärt sökträd

En **sorterad array-lista** har O(log(n)) för sökning och O(n) för insättning.

Den slösar maximalt halva arrayen (genomsitt en fjärdedel) eftersom den dubblar storleken när den är full

Ett **binärt sökträd** har O(log(n)) för insättning och sökning men behöver lagringsutrymme för två länkar per element.

En **hash-tabell** 75% full har extremt mycket bättre prestanda (i princip O(1)) och slösar mindre plats om vi använder öppen adressering. Använder vi länkning får vi bra prestanda och mindre lagringsutrymme än ett binärt sökträd men mer lagringsutrymme jämfört med array-listan. Nackdelen är att den inte ordnar element. Behöver vi detta välj binärt sökträd istället. Tex om vi vill kunna hitta närmaste elementet eller gå igenom alla element (detta är O(size) för hashtabell).

Implementera en hashtabell med länkning

Konstruktorn ska ge oss möjlighet att sätta storleken. Helst ska den sedan välja ett närliggande primtal men det kan vi hoppa.

put(key,value) sätter in key, value på rätt index enligt key's hashkod och returnerar det gamla value om key hittas och null om det inte hittas

get(key) returnerar motsvarande value om key finns annars null

remove(key) tar bort key, value elementet om det finns och returnerar value annars returneras null

Vi använder vår enkellänkade lista för denna implementering. Denna användning av en lista är ett exempel då det är mycket viktigt att inte slösa utrymme i listan då vi har n listor! Så ingen länk tail och helst inte variabel size. En arraylist kräver i snitt mindre lagringsutrymme men har O(m) på att ta bort ett element (länkade listan har O(1)). Observera också att eftersom de flesta listor endast innehåller ett element så om vi väljer arraylist bör vi välja startstorlek till 1.

Implementering

```
public class HashTableBucket<K,V> {

  private static class Entry<K,V>{
    public K key;
    public V value;
    public Entry(K k, V v){
      key=k;
      value=v;
    }
  }

  private SingleLinkedList<Entry<K,V>>[] table;

  @SuppressWarnings("unchecked")
  public HashTableBucket(int size){
    table  = new SingleLinkedList[size];
  }
…
```

```
public V put(K key, V value){
  int index = key.hashCode()%table.length;
  if(index<0) index+= table.length;
  if(table[index]==null){
    table[index] = new SingleLinkedList<Entry<K,V>>();
    table[index].add(new Entry<K,V>(key,value));
    return null;
  }else{
    V oldValue;
    for(Entry<K,V> e: table[index]){
      if(e.key.equals(key)){
        oldValue=e.value;
        e.value = value;
        return oldValue;
      }
    }
    table[index].add(0, new Entry<K,V>(key,value));
    return null;
  }
}
```

Glöm inte att SingleLinkedList ska **implements Iterable<E>** för att for-loopen ska fungera

```
public V get(K key){
  int index = key.hashCode()%table.length;
  if(index<0) index+=table.length;
  if(table[index]==null) return null;
  for(Entry<K,V> e: table[index]){
    if(e.key.equals(key)) return e.value;
  }
  return null;
}
```

Avbildning i JCF

Interface Map<K,V>

implementeras av

Class HashMap<K,V> - är ej trådsäker

Class Hashtable<K,V> - trådsäker.

Båda använder länkning och rehashar då de är 75% fulla.
Läs mer i Java API.

Java byter från lista till binärt sökträd om fler än 8 element på samma index